

MedExpertMatch — Полная презентация (45 мин)

1. Титульный слайд

- Название: MedExpertMatch — AI-система подбора медицинских экспертов
- Изображение: Spring AI Agent Skills Medical (визуальный образ агентной архитектуры)
- by Siarhei Berdachuk
- Стек: Java 21, Spring Boot 4, Spring AI 2.0, PostgreSQL 17 + PgVector + Apache AGE

2. Siarhei Berdachuk

3. Why this project

- Изначально был вопрос: как реально улучшить качество RAG-систем?
- Главная проблема RAG: мусор на входе → мусор на выходе (garbage in, garbage out)
- Гипотеза: если скрестить три типа данных — реляционные, векторные и графовые — получится умнее
- Проверка стека: PostgreSQL + PgVector + Apache AGE + Spring AI + AI-агенты
- Выбор домена: медицина — здесь цена ошибки максимальна, результаты видны сразу
- Цель: создать полноценное работающее приложение, а не просто proof-of-concept
- Главный вопрос: можно ли совместить реляции, векторы, граф, LLM и агентов в одном стеке?
- Код открыт: github.com/berdachuk/med-expert-match-ce

4. Agenda

- 1) Проблемы — что болит у врачей и больницы
- 2) Архитектура — как устроена система изнутри
- 3) Демонстрация прототипа — живой показ
- 4) Вопросы и ответы Q&A
- Логика: сначала проблема → потом решение → потом доказательство

5. Problem 1: Consultation delays (задержки консультаций)

- Что происходит сейчас: чтобы найти нужного узкого специалиста, уходят дни и недели
- Почему: нет единой системы — всё держится на личных контактах и обзвонах
- Последствия: пациент ждёт → риски растут → срок госпитализации увеличивается
- Как решаем: умная очередь с автоматической приоритизацией по срочности и сложности
- Результат: подбор подходящего специалиста за минуты, а не за дни

6. Problem 2: Invisible expertise (невидимая экспертиза)

- Что происходит: никто не знает, кто в больнице лучший по конкретному диагнозу
- Знание сидит в головах у старших врачей, а не в информационной системе
- Отсюда проблемы: пациента направляют не к тому специалисту, нагрузка распределена криво
- Как решаем: строим граф связей врач–диагноз–процедура–исход
- Система видит полную картину: кто, что, когда лечил и с каким результатом
- Плюс сетевая аналитика: выявляем скрытые паттерны и связи между врачами

7. Problem 3: Fragmented decision support (разрозненные инструменты)

- Что происходит: врач сидит и прыгает между 4–5 окнами
- Окно 1: карточка пациента. Окно 2: PubMed / статьи. Окно 3: клинические рекомендации. Окно 4: список врачей
- Это дико неэффективно: контекст теряется, время уходит, ошибки множатся
- Как решаем: единый агентный сценарий из четырёх связанных шагов
- Шаг 1 — анализ кейса. Шаг 2 — поиск доказательной базы. Шаг 3 — рекомендации. Шаг 4 — подбор врача
- Всё в одном окне, контекст не теряется между шагами

8. Problem 4: Suboptimal use of resources (неоптимальное использование ресурсов)

- Что происходит: пациента отправляют в ближайшее учреждение, а не в лучшее
- Но «ближайшее» не равно «подходящее» — там может не быть нужного специалиста
- Если везти вслепую — потеря времени, повторные маршрутизации, лишние затраты
- Как решаем: умная маршрутизация с учётом трёх факторов
- Фактор 1: специализация учреждения. Фактор 2: текущая загрузка. Фактор 3: география
- На выходе — оптимальный маршрут, а не просто ближайшая точка на карте

9. Six core scenarios (шесть сценариев системы)

- Каждый сценарий — это роль пользователя + его задача
- 1) Стационарный матчинг — лечащий врач подбирает консультанта для пациента в отделении
- 2) Второе мнение / телемедицина — направляющий врач ищет удалённого эксперта
- 3) Приоритизация очереди — координатор расставляет пациентов по срочности
- 4) Сетевая аналитика — главврач или СМО видит картину компетенций всей организации
- 5) Региональная маршрутизация — операционный отдел направляет пациентов между учреждениями
- 6) Поддержка решений — клиницист получает анализ кейса + ссылки на исследования
- Два способа входа: загрузка FHIR-пакета (стандартный медицинский формат) или просто текст
- Все сценарии используют одни и те же компоненты, но в разных комбинациях

10. Request path (путь запроса — главный сценарий)

- Одна сквозная нить: MedicalCase сохраняется и векторизуется (PgVector) → Agent API → case-analyzer → doctor-matcher → SemanticGraphRetrievalService считает баллы → ранжированный список с обоснованием в UI/API
- Что происходит на каждом шаге (детализация)
- MedicalCase + PgVector: текст кейса → числовой-эмбеддинг → индекс для быстрого поиска похожих
- Agent API: POST /agent/match — единый вход, за которым прячется вся агентная логика
- case-analyzer: AI-агент извлекает ICD-10, срочность, специализацию → уточняет картину
- doctor-matcher: AI-агент запускает SGR (Semantic Graph Retrieval) — трёхканальный поиск: вектор + граф + ключевые слова
- SGR-скоринг: взвешенное слияние 40% вектор / 30% граф / 30% история — балл от 0 до 1 на каждого врача
- Результат: отсортированный список с текстовым «почему именно этот врач»
- Это якорный сценарий архитектуры — на нём держится вся система

11. Find Specialist flow (детальные шаги)

- Шаг 1: создать или загрузить медицинский кейс → получаем объект MedicalCase
- Шаг 2: автоматически генерируем embedding (векторное представление) для этого кейса
- Шаг 4: case-analyzer уточняет кейс → ICD-10, срочность, специализация
- Шаг 5: doctor-matcher вычисляет score(case, doctor) через SGR (Semantic Graph Retrieval) для каждого врача
- Шаг 6: возвращаем ранжированный список врачей с индивидуальным обоснованием
- Весь процесс от начала до конца занимает секунды

Шаг 3: вызываем POST /api/v1/agent/match/{caselid} — стартует агентный процесс

12. System Overview (архитектура сверху)

- Многоуровневая архитектура: Web → LLM → Retrieval → Domain → Infrastructure
- Event-driven модель: PlannerAgent планирует → ContextBuilderAgent собирает контекст → ExecutionAgent выполняет
- Retrieval слой: DoctorMatchService получает запрос → SGR (Semantic Graph Retrieval) выполняет гибридный поиск → ReRankingService переранжирует
- База данных: PostgreSQL 17 с расширениями pgvector (для векторов) и Apache AGE (для графов)
- Вся аналитика, поиск и графовые запросы — внутри одного экземпляра PostgreSQL

13. Hybrid retrieval: GraphRAG (гибридный поиск)

- Три независимых поисковых канала работают параллельно
- 1) Векторный канал: pgvector cosine similarity — ищет врачей, у которых был похожий опыт
- 2) Графовый канал: Apache AGE Cypher traversal — обходит связи врач–диагноз–процедура
- 3) Ключевые слова: прямое сопоставление кодов МКБ-10 с процедурами врача
- RRF (Reciprocal Rank Fusion) — умное слияние трёх списков без калибровки сырых оценок
- Финальный штрих: семантический rerank — LLM перепроверяет топ результатов
- Почему не только эмбеддинги: похожий текст не гарантирует похожий медицинский исход

14. Graph structure (графовая модель данных)

- Вершины графа: Doctor, MedicalCase, ICD10Code, MedicalSpecialty, Facility
- Основные рёбра (связи):
- TREATED — врач лечил конкретный случай
- CONSULTED_ON — врач консультировал по случаю
- SPECIALIZES_IN — врач имеет специализацию
- TREATS_CONDITION — врач лечит определённое заболевание
- AFFILIATED_WITH — врач привязан к учреждению
- HAS_CONDITION — у случая есть диагноз (код МКБ-10)
- REQUIRES_SPECIALTY — случай требует определённой специализации
- Сборка графа: MedicalGraphBuilderService автоматически строит связи при загрузке данных
- Доступ к графу: только через GraphService (единая точка входа для всех Cypher-запросов)

15. SGR: Semantic Graph Retrieval (как устроен поиск)

- Составной вес врач–кейс: 40% вектор + 30% граф + 30% история
- Граф — это самостоятельный полноценный сигнал, а не просто довесок к эмбеддингам
- Исторический сигнал: реальные рейтинги (1–5 звёзд) + зафиксированные исходы лечения (SUCCESS / IMPROVED / NO_CHANGE)
- Все три сигнала независимы: если один врёт — два других страхуют

16. The Three-Signal Scorer (три источника оценки)

- Каждый врач оценивается по трём независимым измерениям
- Векторный сигнал (40%): pgvector cosine similarity — насколько опыт врача близок к описанию случая
- Графовый сигнал (30%): Apache AGE Cypher traversal — сила связей врач–диагноз–процедура
- Исторический сигнал (30%): ClinicalExperienceOutcomes — реальные результаты прошлых лечений
- Fusion (слияние): два варианта на выбор
- Weighted Average — средневзвешенное с настраиваемыми коэффициентами
- RRF (k=60) — Reciprocal Rank Fusion, Взаимное слияние рангов без калибровки сырых оценок
- После слияния: Semantic Re-ranker — LLM перепроверяет топ результатов
- Все веса настраиваются через application.yml без перекомпиляции

17. Spring Modulith (модульная архитектура)

- Приложение разбито на независимые модули со строгими границами
- core — общая инфраструктура (конфигурация, утилиты, обработка ошибок)
- doctor, medicalcase, medicalcoding, facility — доменные сущности и их репозитории
- caseanalysis — AI-анализ медицинских случаев
- embedding — работа с векторными представлениями в PgVector
- retrieval — гибридный GraphRAG поиск и матчинг
- llm — оркестрация LLM и AI-агентов
- graph — работа с графовой базой Apache AGE
- ingestion — загрузка данных (FHIR, синтетика)
- web — веб-интерфейс на Thymeleaf
- Границы enforced на уровне компиляции: модуль А не может тайком использовать модуль Б без разрешения
- llm — оркестратор сквозных медицинских сценариев, связывает все модули вместе

18. LLM orchestration and tools (как работают AI-агенты)

- MedicalAgentService — центральный оркестратор, запускает рабочие процессы
- 9 классов инструментов, в каждом по несколько @Tool-методов, всего 27 инструментов
- Главное правило: LLM никогда не ходит напрямую в базу данных — только через инструменты
- AutoMemoryTools — память агента между разными сессиями (что уже находили, что предлагали)
- SessionMemoryAdvisor — автоматическое сжатие истории диалога после 15 шагов (хранится в JDBC)
- Все промпты вынесены во внешние .st файлы (StringTemplate4) — можно менять без перекомпиляции

19. Nine agent skills (девять AI-навыков)

- 1) case-analyzer — извлекает из описания случая сущности, коды ICD-10, уровень срочности
- 2) doctor-matcher — запускает SGR-матчинг (Semantic Graph Retrieval); ищет лучших врачей под конкретный случай
- 3) evidence-retriever — ищет научные статьи в PubMed и клинические руководства
- 4) recommendation-engine — формирует итоговые рекомендации на основе анализа и evidence
- 5) clinical-advisor — помогает с дифференциальной диагностикой и оценкой рисков
- 6) network-analyzer — анализирует граф: кто с кем связан, где проблемы в компетенциях
- 7) routing-planner — строит оптимальный маршрут между учреждениями
- 8) clinical-guideline — работает с формальными клиническими рекомендациями (CPG + GRADE)
- 9) triage — быстрая оценка срочности: CRITICAL / HIGH / MEDIUM / LOW

20. How Skills Work (как работают навыки)

- Discovery (обнаружение): SkillsTool сканирует все .md файлы навыков → строит реестр (название + описание)
- Activation (активация): когда приходит запрос, LLM выбирает подходящий навык по смыслу → загружается полный текст SKILL.md
- Execution (выполнение): модель вызывает @Tool-методы из загруженного навыка → те дергают реальные сервисы
- Важное ограничение: скрипты навыков работают без sandbox, но в режиме read-only для мутирующих операций
- Это не black-box magic — каждый шаг логируется и воспроизводится

21. Data access: graph and SQL (как система работает с данными)

- Графовая часть: все Cypher-запросы проходят только через GraphService
- Это единая точка входа для работы с графом Apache AGE
- Так проще аудировать: видно кто, когда и зачем лазил в граф
- SQL-часть: JDBC-репозитории, запросы лежат в отдельных .sql файлах
- Пример Cypher-запроса (поиск врачей, лечивших конкретный диагноз):
- MATCH (d:Doctor)-[:TREATED]->(c:MedicalCase)-[:HAS_CONDITION]->(i:ICD10Code)
- WHERE i.code = 'E11.9' RETURN d
- Аналогично работают связи: SPECIALIZES_IN, TREATS_CONDITION, AFFILIATED_WITH

22. Ingestion and FHIR (загрузка медицинских данных)

- Модуль ingestion отвечает за приём и подготовку данных
- FHIR-адатеры: Patient, Condition, Encounter, Observation — стандартные медицинские форматы
- Для тестирования и демонстрации генерируем синтетические данные (реалистичные, но обезличенные)
- После загрузки данных автоматически запускается: генерация эмбеддингов + построение графа связей
- Вся елочка: загрузили → проиндексировали → готовы к поиску

23. Privacy, deployment and data (приватность и развёртывание)

- Шесть сценариев использования: стационар, очередь, второе мнение, маршрутизация, аналитика, поддержка
- Демонстрационные данные: только синтетика, никаких реальных пациентов, никакого PHI
- Технологический стек: Spring Boot 4.1, PostgreSQL 17, PgVector, Apache AGE, Thymeleaf, OpenAI API
- Медицинские дисклеймеры встроены во все выводы LLM — система не заменяет врача
- Профили запуска: local (для разработки), demo (для презентации)

24. Harness — idea (управляющая обвязка)

- Основная идея: модель предлагает, harness направляет и ограничивает
- Java-оркестрация на каждом шаге: routing (куда идти), context (что передать), tools (чем работать), verify (проверить)
- Результат: надёжные, воспроизводимые медицинские рабочие процессы
- Вместо подхода «надемся, что промпт работает» — детерминированная оркестрация

25. Harness — workflow (как работает процесс)

- Конвейер из пяти состояний: PLANNING → CONTEXT_BUILT → TOOLS_EXECUTED → VERIFYING → POLICY_GATE → DONE
- Цикл проверки (VERIFYING): если инструмент не сработал — повторяем (по умолчанию до 2 проходов)
- Policy gate — финальный барьер: проверка на PHI, наличие дисклеймера, корректность вывода
- Принцип fail-closed: если проверка не прошла — лучше не отвечать, чем ответить небезопасно
- GoalClassifier: определяет тип запроса (match / analyze / route) ещё до запуска движка — экономит ресурсы

26. Agent vs chat — ROI test (сравнение подходов)

- Четыре слоя системы: Chat → Harness → Policy → Data flywheel
- Правило Go/No-Go для включения агентного режима: качество должно вырасти на 20% при росте стоимости не более чем в 2 раза
- Expert match через harness: GraphRAG + MedGemma даёт прирост качества ценой ~2–3× большего расхода токенов
- Вывод: агентный режим оправдан для сложных случаев, простые идут через быстрый chat

27. FunctionGemma — tool calling (специализированная модель)

- Две специализированные модели вместо одной универсальной
- MedGemma 1.5:4b — «думающая» модель: медицинские рассуждения, анализ, выводы
- FunctionGemma 270m — «исполняющая» модель: только структурированные выавы @Tool-методов
- Когда запрос идёт через harness — FunctionGemma пропускается (не нужна, хватает оркестрации)
- В режиме Auto chat: FunctionGemma подключается для вызова инструментов (evidence, edge cases)

28. Demo: preparation (подготовка к демонстрации)

- База данных: PostgreSQL 17 с расширениями PgVector + AGE, профиль demo
- Запуск: Docker-контейнер со всем необходимым внутри
- Датасет: medium — достаточно данных для показательной демонстрации
- План Б: скрипты или запись видео на случай, если live-демо подведёт

29. Conclusion (выводы)

- Это не просто прототип — это полноценный стековый эксперимент
- Проверили гипотезу: PostgreSQL + PgVector + AGE + Spring AI + агенты — работают вместе
- Решённые проблемы: задержки консультаций, скрытая экспертиза, разрозненные инструменты, неоптимальные ресурсы
- Четыре архитектурные опоры: GraphRAG + SGR, Modulith, агент + инструменты, политика данных
- Далее: вопросы и ответы Q&A

30. Thank you (заключительный слайд)

- Спасибо за внимание. Вопросы?
- Ссылки: Use Cases, Architecture, Demo Guide
- Сайт: berdachuk.com
- Код: github.com/berdachuk/med-expert-match-ce
- Формат: короткие вопросы на сцене, глубокие технические — после презентации